Q2: Signal Sampling & 8-Point Moving-Average FIR Filter (ATmega32)

Exam Specification: F_CPU = 16 MHz • Vcc = 5 V • ADC Ref = 5 V • Input: 1 V Peak + 2.5 V DC offset (1.5 V to 3.5 V) on ADC0 • Sample Rate: 2 kHz

Step 1: Timer Settings & ADC Prescaler Derivations

1. Timer0 CTC Calculation for 2 kHz Sampling:

• Sampling period $T_s = 1 / f_s = 1 / 2000$ Hz = **0.5 ms = 500 µs**.

• With CPU clock F_CPU = 16 MHz, choose **Prescaler N = 64** (CS01=1, CS00=1).

• Timer tick frequency: $f_{timer} = 16$ MHz / 64 = 250,000 Hz (tick period = 4 µs).

• Required counts to reach 500 µs: OCR0 = $(f_{timer} / f_s) - 1 = (250{,}000 / 2{,}000) - 1 = 125 - 1 =$ **124**.

• *Justification:* 124 easily fits in 8-bit register OCR0 (0–255) and gives an *exact integer match* with zero timing error.

2. ADC Prescaler Justification:

• ATmega32 successive approximation ADC requires an input clock between **50 kHz and 200 kHz** for full 10-bit accuracy (up to 1 MHz for 8-bit).

• With 16 MHz clock, Prescaler options: 64 → 16MHz/64 = 250 kHz; **Prescaler 128 → 16MHz/128 = 125 kHz** (ADPS2:0 = 111).

• At 125 kHz, one standard conversion takes 13 ADC clock cycles = 13 × 8 µs = **104 µs**.

• Since 104 µs < 500 µs (the 2 kHz sampling interval), the conversion finishes well before the next sample tick, guaranteeing zero jitter and no sample dropping.

Step 2: Complete Embedded C Implementation (ATmega32)

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>

#define WINDOW 8
volatile uint16_t raw_x = 0;
volatile uint16_t filtered_y = 0;
volatile uint8_t sample_ready = 0;

// 8-Point Circular Buffer for FIR
uint16_t ring_buf[WINDOW] = {0};
uint8_t buf_idx = 0;
uint32_t running_sum = 0;

void timer0_init_2khz(void) {
    TCCR0 = (1 << WGM01) | (1 << CS01) | (1 << CS00); // CTC mode, Prescaler 64
    OCR0 = 124;                                       // 2 kHz interrupt (500 us)
    TIMSK |= (1 << OCIE0);                            // Enable Timer0 compare match
}

void adc_init(void) {
    ADMUX = (1 << REFS0);                             // AVCC reference (5V), Channel ADC0 (PA0)
    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0); // Enable, Prescaler 128 (125 kHz)
}

void uart_init(uint16_t ubrr_val) {
    UBRRH = (uint8_t)(ubrr_val >> 8);
    UBRRL = (uint8_t)ubrr_val;
    UCSRB = (1 << TXEN);
    UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0); // 8-data bits, 1 stop bit
}

void uart_send_char(char c) {
    while (!(UCSRA & (1 << UDRE)));
    UDR = c;
}

void uart_send_num(uint16_t val) {
    char buf[6]; int i = 0;
    if (val == 0) { uart_send_char('0'); return; }
    while (val > 0) { buf[i++] = (val % 10) + '0'; val /= 10; }
    while (i > 0) uart_send_char(buf[--i]);
}

// Timer0 fires every 500 us (2 kHz) to sample and filter
ISR(TIMER0_COMP_vect) {
    ADCSRA |= (1 << ADSC);                            // Start conversion
    while (ADCSRA & (1 << ADSC));                     // Wait ~104 us
    raw_x = ADC;                                      // Read 10-bit raw sample

    // 8-Point Moving Average FIR: Update running sum & divide by 8 (shift right 3)
    running_sum -= ring_buf[buf_idx];
    ring_buf[buf_idx] = raw_x;
    running_sum += raw_x;
    buf_idx = (buf_idx + 1) & 7;                      // Modulo 8 wrap-around
    filtered_y = (uint16_t)(running_sum >> 3);        // Fast division by 8

    sample_ready = 1;
}

int main(void) {
    adc_init();
    timer0_init_2khz();
    uart_init(103); // 9600 Baud @ 16 MHz: UBRR = (16000000/(16*9600))-1 = 103
    sei();

    while (1) {
        if (sample_ready) {
            sample_ready = 0;
            // Transmit CSV: raw,filtered\r\n to SimulIDE Serial Monitor
            uart_send_num(raw_x);
```

Step 3: Complete AVR Assembly Implementation (Pure Assembly)

Using GNU-syntax / AVR-Assembler for ATmega32. Implements Timer0 ISR, ADC conversion, 8-element RAM buffer, and arithmetic shift right 3 for division by 8.

```
; ====================================================================
; ATmega32 Signal Sampling (2 kHz) & 8-Point Moving Average FIR in Assembly
; ====================================================================
.include "m32def.inc"

.dseg
.org 0x0060              ; Internal SRAM start on ATmega32
buffer:      .byte 16    ; 8 samples x 2 bytes (16-bit entries) = 16 bytes
buf_ptr:     .byte 1     ; Index pointer 0..7

.cseg
.org 0x0000
    rjmp RESET           ; Reset vector
.org 0x0014
    rjmp TIMER0_COMP_ISR ; Timer0 Compare Match Handler vector on ATmega32

RESET:
    ; 1. Initialize Stack Pointer
    ldi r16, high(RAMEND)
    out SPH, r16
    ldi r16, low(RAMEND)
    out SPL, r16

    ; 2. Clear RAM Buffer
    ldi ZL, low(buffer)
    ldi ZH, high(buffer)
    ldi r17, 16
    clr r16
CLR_LOOP:
    st Z+, r16
    dec r17
    brne CLR_LOOP
    sts buf_ptr, r16

    ; 3. Initialize ADC: AVCC (REFS0=1), Prescaler 128 (ADEN=1, ADPS2:0=111)
    ldi r16, (1 << REFS0)
    out ADMUX, r16
    ldi r16, (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0)
    out ADCSRA, r16

    ; 4. Initialize Timer0: CTC mode (WGM01=1), Prescaler 64 (CS01=1, CS00=1)
    ldi r16, (1 << WGM01) | (1 << CS01) | (1 << CS00)
    out TCCR0, r16
    ldi r16, 124         ; OCR0 = 124 for 2 kHz sampling
    out OCR0, r16
    in r16, TIMSK
    sbr r16, (1 << OCIE0)
    out TIMSK, r16

    ; 5. Initialize UART Transmitter (9600 Baud @ 16 MHz: UBRR = 103)
    clr r16
    out UBRRH, r16
    ldi r16, 103
    out UBRRL, r16
    ldi r16, (1 << TXEN)
    out UCSRB, r16
    ldi r16, (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0)
    out UCSRC, r16

    sei                  ; Global Interrupt Enable
MAIN_LOOP:
    rjmp MAIN_LOOP

; --- Timer0 ISR (Triggered every 500 us) ---
TIMER0_COMP_ISR:
    push r16
    in r16, SREG
    push r16
    push r17
    push r18
    push r19
    push r20
    push r21
```

Step 4 & 5: Frequency Response, Aliasing Analysis & 8 MHz Clock Change

Input Signal	Filter Frequency Response H(f)	Expected Output Behavior on Serial / Scope
50 Hz Sine	$f / f_s = 50 / 2000 = 0.025$ (very low frequency). Passband gain $\|H(50)\| \approx 0.98$.	Signal passes almost entirely unattenuated. Output tracks the 50 Hz wave smoothly with noise removed.
500 Hz Sine	$f / f_s = 500 / 2000 = 0.25 = 1/4$. Moving average FIR has transmission zeroes at multiples of $f_s/M = 2000/8 =$ **250 Hz**.	At 500 Hz, the filter is exactly at its **second transmission notch (zero)**! The 500 Hz sinusoidal component is **heavily suppressed / cancelled to zero amplitude**, leaving only the DC offset (2.5 V).
1.5 kHz Sine (Aliasing)	$f_{in} = 1500$ Hz > $f_{Nyquist}$ (1000 Hz). Apparent Aliased Frequency: $\mathbf{f_{apparent} = \|f_s - f_{in}\| = \|2000 - 1500\| = 500}$ **Hz**.	The ADC samples create an erroneous 500 Hz wave. Because the 8-point filter places a zero at 500 Hz, the aliased artifact will also be attenuated, but the original signal is irreparably destroyed.

Critical Exam Answers:

1. Why can't a digital filter applied after sampling correct aliasing?

Once an analog signal is sampled below the Nyquist rate ($f_s < 2 \cdot f_{in}$), high-frequency spectral copies fold directly on top of the baseband spectrum (frequency folding). At that point, the digitized discrete data points of a 1.5 kHz wave are mathematically indistinguishable from a legitimate 500 Hz wave. A digital filter operates purely on discrete numbers and cannot distinguish an aliased signal from a true baseband signal. *Aliasing can only be prevented before sampling using an analog anti-aliasing low-pass filter (cutoff ≤ $f_s/2$).*

2. What happens if we change the CPU clock from 16 MHz to 8 MHz?

• **Timer & Sampling Rate Halves:** With prescaler 64 and OCR0 = 124 unchanged, tick clock drops from 250 kHz to 125 kHz. The sampling rate becomes $f_s = 125{,}000 / 125 =$ **1 kHz (T_s = 1 ms)** instead of 2 kHz.

• **ADC Conversion Time Doubles:** With Prescaler 128, ADC clock drops to 8 MHz / 128 = 62.5 kHz (within the 50–200 kHz spec, but conversion duration increases from 104 µs to **208 µs**).

• **UART Baud Rate Halves:** Baud rate drops from 9600 to **4800 baud** unless UBRR is adjusted to `(8000000/(16*9600)) - 1 = 51`.

• *To maintain the exact 2 kHz sampling rate at 8 MHz:* Change OCR0 to **61** with prescaler 64: `OCR0 = (8000000 / (64 * 2000)) - 1 = 62.5 - 1 = 61.5`, or better, set **Prescaler = 32** (if supported) or **Prescaler = 8** with OCR0 = `(8,000,000 / (8 * 2,000)) - 1 = 499` (using 16-bit Timer1).